Introduction

Welcome! In this assignment, we will build a Convolutional Neural Network (CNN) to classify handwritten digits from the famous MNIST dataset. This dataset is a classic in the field of computer vision and provides a great starting point for understanding image classification with deep learning.

Section 1: Setting Up - Imports

Before we dive into building our CNN, we need to import the necessary libraries. These libraries provide pre-built tools and functions that will make our work much easier.

Instructions:

1. **Carefully review the code cell below.** It imports libraries from TensorFlow and Keras, which are powerful frameworks for building and training neural networks.
2. **Execute the code cell by selecting it and pressing [Shift + Enter] (or the "Run" button).**
3. **Ensure there are no error messages after running the cell.** If you encounter errors, double-check that you have TensorFlow and Keras installed in your environment.

```
# Cell 1: Imports
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
display("Imports complete.")
```

```
'Imports complete.'
```

Explanation of Imports:

- **`tensorflow as tf` and `keras`:** TensorFlow is the main deep learning framework, and Keras is its high-level API that simplifies building and training models. We import TensorFlow as `tf` and Keras directly for easy access to their functionalities.
- **`from tensorflow.keras import layers`:** This imports the `layers` module from Keras, which provides various layers for building neural networks (like convolutional layers, dense layers, etc.).
- **`from tensorflow.keras.datasets import mnist`:** This imports the MNIST

dataset directly from Keras datasets. This is very convenient for loading and using the MNIST data.

- `from tensorflow.keras.utils import to_categorical`: This imports the `to_categorical` function, which we will use to perform one-hot encoding of our labels.

Section 2: Data Loading and Preprocessing

In this section, we will load the MNIST dataset and prepare it for training our CNN model. Preprocessing steps are crucial to ensure our data is in the right format for the model to learn effectively.

```
# Cell 2: Data Loading and Preprocessing
# Load the MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize pixel values to be between 0 and 1
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Add a channel dimension (for grayscale images, it's 1)
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

# One-hot encode the labels
num_classes = 10
y_train = to_categorical(y_train, num_classes)
y_test = to_categorical(y_test, num_classes)
display("Data loading and preprocessing complete.")
```

```
'Data loading and preprocessing complete.'
```

Section 7: Reflection Questions

Reflection Questions:

1. **Conv2D Layer:** What is the role of the Conv2D layer? How do the `kernel_size` and the number of filters affect the learning process? *Hint: Experiment by changing these values in Cell 3.*
2. **MaxPooling2D Layer:** What is the purpose of the MaxPooling2D layer? How does it contribute to the model's performance? *Hint: Try removing or adding a MaxPooling2D layer and see what happens.*
3. **One-Hot Encoding:** Why do we use one-hot encoding for the labels?

4. **Flatten Layer:** Why do we need the Flatten layer before the Dense layer?
5. **Optimizer and Loss Function:** What optimizer and loss function did you choose in Cell 4? Explain your choices. Why is categorical cross-entropy a suitable loss function for this task? Why is Adam a good choice of optimiser?
6. **Batch Size and Epochs:** How did you choose the batch size and number of epochs in Cell 5? What are the effects of changing these parameters? *Hint: Experiment!*
7. **Dropout:** Why is the Dropout layer included in the model?
8. **Model Architecture:** Describe the overall architecture of your CNN. How many convolutional layers did you use? How many max pooling layers? What is the final dense layer doing?
9. **Performance:** What accuracy did you achieve on the test set? Are you happy with the result? Why or why not? If you're not happy, what could you try to improve the performance?

Tips and Explanations:

- **Normalization:** Dividing the pixel values by 255 normalizes them to the range [0, 1]. This is important for training neural networks.
- **Reshaping:** The `reshape` operation adds a channel dimension to the images. For grayscale images, the channel dimension is 1.
- **One-Hot Encoding:** `to_categorical` converts the class labels (0-9) into one-hot encoded vectors.
- **Conv2D Parameters:** The `kernel_size` determines the size of the convolutional filter (e.g., 3x3). The number of filters determines how many different features are learned.
- **MaxPooling2D Parameters:** The `pool_size` determines the size of the pooling window (e.g., 2x2).
- **Optimizer:** The optimizer is the algorithm used to update the model's weights during training.
- **Loss Function:** The loss function measures the error between the model's predictions and the true labels.
- **Batch Size:** The batch size is the number of samples processed in each training iteration.
- **Epochs:** An epoch is one complete pass through the entire training dataset.
- **Dropout:** Dropout is a regularization technique that helps prevent overfitting.

Note on Performance:

The test accuracy in Cell 6 is quite low (around 10%). This is unexpected for the MNIST

dataset with this model architecture. Here are some potential reasons and things to investigate:

- **Data Issues:** Double-check that the data loading and preprocessing steps in Cell 2 are correct. Ensure that the `x_test` and `y_test` variables are loaded and preprocessed properly.
- **Model Architecture:** While the architecture is a good starting point, you could experiment with:
    - Adding more convolutional layers.
    - Increasing the number of filters in the convolutional layers.
    - Using different activation functions.
    - Adjusting the dropout rate.
- **Training Parameters:**
    - Increase the number of epochs to allow the model to train for longer.
    - Experiment with different batch sizes.
    - Try a different optimizer.
- **Overfitting/Underfitting:** Analyze the training and validation accuracy and loss from the output of Cell 5.
    - If training accuracy is high but validation accuracy is low, the model might be overfitting. You could try increasing dropout or adding more regularization.
    - If both training and validation accuracy are low, the model might be underfitting. You could try making the model more complex (e.g., adding more layers or filters) or training for more epochs.

By experimenting with these aspects, you should be able to significantly improve the test accuracy.

Explanation of Data Preprocessing:

- **Loading the MNIST dataset:** `mnist.load_data()` loads the MNIST dataset, which is already split into training and testing sets (`(x_train, y_train), (x_test, y_test)`). `x_train` and `x_test` contain the images (pixel data), and `y_train` and `y_test` contain the corresponding labels (digits 0-9).
- **Normalization:** `x_train = x_train.astype("float32") / 255.0` and `x_test = x_test.astype("float32") / 255.0` normalize the pixel values. Pixel values in images are typically in the range 0-255. Dividing by 255 scales them to the range 0-1. This normalization helps the neural network train faster and more effectively.

- **Adding Channel Dimension:** `x_train = x_train.reshape(-1, 28, 28, 1)` and `x_test = x_test.reshape(-1, 28, 28, 1)` reshape the data to add a channel dimension. Even though MNIST images are grayscale (single channel), CNNs in Keras expect input data to have a channel dimension. We reshape from `(number_of_images, 28, 28)` to `(number_of_images, 28, 28, 1)`. The `-1` in `reshape` means "infer the dimension based on the size of the array."
- **One-Hot Encoding:** `y_train = to_categorical(y_train, num_classes)` and `y_test = to_categorical(y_test, num_classes)` perform one-hot encoding on the labels. Instead of representing the digit '3' as a single number, one-hot encoding converts it into a vector `[0, 0, 0, 1, 0, 0, 0, 0, 0, 0]`, where the 4th position (index 3) is 'hot' (value 1), and all other positions are 'cold' (value 0). This is a standard way to represent categorical labels for neural networks in multi-class classification problems. `num_classes = 10` specifies that we have 10 classes (digits 0-9).

Section 3: Model Definition - Building the CNN

Now we will define the architecture of our Convolutional Neural Network (CNN). You will be building a sequential model using Keras layers.

```
# Cell 3: Model Definition
# Build the CNN model.
model = keras.Sequential(
    [
        keras.Input(shape=(28, 28, 1)),  # Input layer
        layers.Conv2D(32, kernel_size=(3, 3), activation="relu"), # Convolution
        layers.MaxPooling2D(pool_size=(2, 2)), # Max pooling layer 1
        # Students: Add another Conv2D layer here.  Experiment with the number
        # layers.Conv2D(____, kernel_size=(____, ____), activation="____"),  #
        layers.Conv2D(64, kernel_size=(3, 3), activation="relu"), # Convolution
        layers.MaxPooling2D(pool_size=(2, 2)), # Max pooling layer 2
        # Students: Add another MaxPooling2D layer here if needed.
        # layers.MaxPooling2D(pool_size=(____, ____)),  # Max pooling layer 2
        layers.Flatten(),  # Flatten layer
        layers.Dropout(0.5),  # Dropout layer
        layers.Dense(num_classes, activation="softmax"),  # Output layer
    ]
)
model.summary()
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
conv2d_6 (Conv2D)	(None, 26, 26, 32)	320

conv2d_6 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_6 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_7 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_7 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten_3 (Flatten)	(None, 1600)	0
dropout_3 (Dropout)	(None, 1600)	0
dense_3 (Dense)	(None, 10)	16,010

Total params: 34,826 (136.04 KB)
Trainable params: 34,826 (136.04 KB)
Non-trainable params: 0 (0.00 B)

Explanation of Layers:

- **`keras.Input(shape=(28, 28, 1))`:** This is the input layer of our model. It specifies the shape of the input images, which are 28x28 pixels with 1 channel (grayscale).
- **`layers.Conv2D(32, kernel_size=(3, 3), activation="relu")`:** This is a 2D Convolutional layer.
  - `32`: This is the number of filters (also called kernels). Each filter learns to detect specific features in the input image.
  - `kernel_size=(3, 3)`: This defines the size of the convolutional filter as 3x3 pixels.
  - `activation="relu"`: ReLU (Rectified Linear Unit) is the activation function. It introduces non-linearity into the model, allowing it to learn complex patterns.
- **`layers.MaxPooling2D(pool_size=(2, 2))`:** This is a Max Pooling layer.
  - `pool_size=(2, 2)`: It reduces the spatial dimensions of the feature maps by taking the maximum value within each 2x2 window. This helps to reduce the number of parameters, control overfitting, and make the model more robust to small shifts and distortions in the input.
- **`layers.Flatten()`:** This layer flattens the 2D feature maps from the convolutional and pooling layers into a 1D vector. This is necessary to connect the convolutional part of the network to the fully connected (Dense) layers.
- **`layers.Dropout(0.5)`:** This is a Dropout layer.
  - `0.5`: This sets the dropout rate to 50%. During training, this layer randomly sets 50% of the input units to 0 at each update. This is a regularization

technique that helps to prevent overfitting.

- **`layers.Dense(num_classes, activation="softmax")`:** This is the output Dense (fully connected) layer.
  - `num_classes`: This is set to 10 because we have 10 classes (digits 0-9).
  - `activation="softmax"`: Softmax activation ensures that the output values are probabilities, and they sum up to 1 across all classes. The output will be a vector of 10 probabilities, where each probability represents the model's confidence that the input image belongs to that specific digit class.

Section 4: Model Compilation - Choosing Loss and Optimizer

Before we can train our model, we need to compile it. Compilation involves choosing an optimizer, a loss function, and metrics to evaluate the model's performance.

```
# Cell 4: Model Compilation
# : Choose an appropriate loss function and optimizer.  Why did you choose thes
model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accu
display("Model compilation complete.")
```

```
'Model compilation complete.'
```

- **Loss Function:** You need to choose a loss function that is appropriate for multi-class classification. Think about what kind of error we are trying to minimize when classifying digits into 10 categories.
- **Optimizer:** You need to choose an optimizer that will efficiently update the model's weights to minimize the loss function. Consider common optimizers used in deep learning.
- **Metrics:** We are using "accuracy" as a metric to evaluate the model's performance. Accuracy is a common metric for classification tasks, representing the percentage of correctly classified images.

Section 5: Model Training - Fitting the Model to the Data

Now it's time to train our CNN model using the training data. Training involves feeding the training data to the model and adjusting its weights to minimize the loss function.

```
# Cell 5: Model Training
# : Adjust the batch size and number of epochs.  to see whatt  happens if you c
history = model.fit(x_train, y_train, batch_size=128, epochs=15, validation_spl
```

```
history = model.fit(x_train, y_train, batch_size=128, epochs=15, validation_spl
display(history.history)
```

```
Epoch 1/15
422/422 ━━━━━━━━━━━━━━━━━━━━ 42s 98ms/step - accuracy: 0.7640 - loss: 0.7665 - v
Epoch 2/15
422/422 ━━━━━━━━━━━━━━━━━━━━ 85s 106ms/step - accuracy: 0.9638 - loss: 0.1206 -
Epoch 3/15
422/422 ━━━━━━━━━━━━━━━━━━━━ 79s 99ms/step - accuracy: 0.9727 - loss: 0.0871 - v
Epoch 4/15
422/422 ━━━━━━━━━━━━━━━━━━━━ 81s 98ms/step - accuracy: 0.9783 - loss: 0.0715 - v
Epoch 5/15
422/422 ━━━━━━━━━━━━━━━━━━━━ 83s 101ms/step - accuracy: 0.9802 - loss: 0.0644 -
Epoch 6/15
422/422 ━━━━━━━━━━━━━━━━━━━━ 81s 98ms/step - accuracy: 0.9817 - loss: 0.0600 - v
Epoch 7/15
422/422 ━━━━━━━━━━━━━━━━━━━━ 41s 97ms/step - accuracy: 0.9837 - loss: 0.0506 - v
Epoch 8/15
422/422 ━━━━━━━━━━━━━━━━━━━━ 82s 98ms/step - accuracy: 0.9853 - loss: 0.0466 - v
Epoch 9/15
422/422 ━━━━━━━━━━━━━━━━━━━━ 82s 97ms/step - accuracy: 0.9853 - loss: 0.0463 - v
Epoch 10/15
422/422 ━━━━━━━━━━━━━━━━━━━━ 82s 97ms/step - accuracy: 0.9872 - loss: 0.0411 - v
Epoch 11/15
422/422 ━━━━━━━━━━━━━━━━━━━━ 82s 97ms/step - accuracy: 0.9875 - loss: 0.0386 - v
Epoch 12/15
422/422 ━━━━━━━━━━━━━━━━━━━━ 82s 98ms/step - accuracy: 0.9881 - loss: 0.0362 - v
Epoch 13/15
422/422 ━━━━━━━━━━━━━━━━━━━━ 81s 97ms/step - accuracy: 0.9889 - loss: 0.0341 - v
Epoch 14/15
422/422 ━━━━━━━━━━━━━━━━━━━━ 42s 99ms/step - accuracy: 0.9880 - loss: 0.0337 - v
Epoch 15/15
422/422 ━━━━━━━━━━━━━━━━━━━━ 81s 97ms/step - accuracy: 0.9892 - loss: 0.0336 - v
{'accuracy': [0.8892777562141418,
  0.9667037129402161,
  0.9740926027297974,
  0.9785370230674744,
  0.9807407259941101,
  0.9825925827026367,
  0.9840370416641235,
  0.985370397567749,
  0.9853518605232239,
  0.9862962961196899,
  0.9873148202896118,
  0.9878518581390381,
  0.9881851673126221,
  0.9883148074150085,
  0.9896110892295837],
 'loss': [0.36803901195526123,
  0.10961253196001053,
  0.08373568952083588,
  0.06981346011161804,
  0.06260359287261963,
  0.057354677468538284,
  0.050111930817365646,
```

```
 0.04590063914656639,
 0.04625527188181877,
 0.04346782714128494,
 0.04003881290555,
 0.037385035306215286,
 0.03588872030377388,
 0.035057730972766876,
 0.03251019865274429],
'val_accuracy': [0.9763333201408386,
 0.9854999780654907,
 0.9868333339691162,
 0.9890000224113464,
 0.9891666769981384,
 0.9900000095367432,
 0.9906666874885559,
 0.9908333420753479,
 0.9913333058357239,
 0.9908333420753479,
 0.9929999709129333,
 0.9915000200271606,
 0.9923333525657654,
 0.9913333058357239,
 0.9931666851043701],
'val_loss': [0.08339144289493561,
 0.055649735033512115,
 0.04719216376543045,
 0.04076939821243286,
 0.041483938694000244,
 0.03712761029601097,
 0.035743702203035355,
 0.03537607938051224,
 0.032492637634277344,
 0.0316389724612236,
 0.027275344356894493,
 0.028237810358405113,
 0.026673976331949234,
 0.028098154813051224,
```

Explanation of Training Parameters:

- **`batch_size`:** This determines the number of training samples processed in each mini-batch during training. A larger batch size can speed up training but might require more memory. A smaller batch size can lead to more noisy updates but might generalize better.
- **`epochs`:** One epoch represents one complete pass through the entire training dataset. More epochs can potentially lead to better training but also increase the risk of overfitting, where the model learns the training data too well and performs poorly on unseen data.
- **`validation_split=0.1`:** This reserves 10% of the training data as a validation set. During training, the model's performance is evaluated on this validation set

set. During training, the model's performance is evaluated on this validation set after each epoch. This helps to monitor for overfitting and tune hyperparameters.

Section 6: Model Evaluation - Assessing Performance on Test Data

After training, we need to evaluate our model's performance on the test dataset. This gives us an estimate of how well the model generalizes to unseen data.

```
# Cell 6: Model Evaluation
loss, accuracy = model.evaluate(x_test, y_test, verbose=0)
print(f"Test loss: {loss:.4f}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Test loss: 0.0251
Test accuracy: 0.9912
```

Section 7: Reflection Questions

Reflection Questions:

1. **Conv2D Layer:** What is the role of the Conv2D layer? How do the `kernel_size` and the number of filters affect the learning process? *Hint: Experiment by changing these values in Cell 3.*
2. **MaxPooling2D Layer:** What is the purpose of the MaxPooling2D layer? How does it contribute to the model's performance? *Hint: Try removing or adding a MaxPooling2D layer and see what happens.*
3. **One-Hot Encoding:** Why do we use one-hot encoding for the labels?
4. **Flatten Layer:** Why do we need the Flatten layer before the Dense layer?
5. **Optimizer and Loss Function:** What optimizer and loss function did you choose in Cell 4? Explain your choices. Why is categorical cross-entropy a suitable loss function for this task? Why is Adam a good choice of optimiser?
6. **Batch Size and Epochs:** How did you choose the batch size and number of epochs in Cell 5? What are the effects of changing these parameters? *Hint: Experiment!*
7. **Dropout:** Why is the Dropout layer included in the model?
8. **Model Architecture:** Describe the overall architecture of your CNN. How many convolutional layers did you use? How many max pooling layers? What is the final

convolutional layers did you use? How many max pooling layers? What is the final dense layer doing?

9. **Performance:** What accuracy did you achieve on the test set? Are you happy with the result? Why or why not? If you're not happy, what could you try to improve the performance?

Tips and Explanations:

- **Normalization:** Dividing the pixel values by 255 normalizes them to the range [0, 1]. This is important for training neural networks.
- **Reshaping:** The `reshape` operation adds a channel dimension to the images. For grayscale images, the channel dimension is 1.
- **One-Hot Encoding:** `to_categorical` converts the class labels (0-9) into one-hot encoded vectors.
- **Conv2D Parameters:** The `kernel_size` determines the size of the convolutional filter (e.g., 3x3). The number of filters determines how many different features are learned.
- **MaxPooling2D Parameters:** The `pool_size` determines the size of the pooling window (e.g., 2x2).
- **Optimizer:** The optimizer is the algorithm used to update the model's weights during training.
- **Loss Function:** The loss function measures the error between the model's predictions and the true labels.
- **Batch Size:** The batch size is the number of samples processed in each training iteration.
- **Epochs:** An epoch is one complete pass through the entire training dataset.
- **Dropout:** Dropout is a regularization technique that helps prevent overfitting.

Conclusion and Submission

we have successfully built and trained a Convolutional Neural Network to classify handwritten digits from the MNIST dataset. we explored key concepts like convolutional layers, pooling layers, activation functions, optimizers, loss functions, and training procedures.